Staj 12. Gün

12. Gün Staj Görevi

Docker + LangChain + Qdrant ile PDF Tabanlı RAG Sistemi

Projenin Amacı

Bu projede amacımız, seçtiğimiz PDF dosyalarını yapay zekânın kullanabileceği bir bilgi tabanına dönüştürmek ve kullanıcıların bu dokümanlar hakkında doğal dilde soru sorabildiği bir RAG sistemi geliştirmekti. Kullanıcı PDF yüklemeyecekti; PDF'ler geliştirici tarafından sisteme önceden eklenecek, kullanıcı ise yalnızca Gradio arayüzünden soru soracaktı.

Projede Kullandığımız Teknolojiler

Bu projede her teknoloji belirli bir görevi üstlendi.

- **Docker:** Uygulamayı ve Qdrant veritabanını container ortamında çalıştırmak.
- **Docker Compose:** Birden fazla container'ı aynı ağ üzerinde yönetmek.
- **LangChain:** PDF işleme ve RAG akışını yönetmek.
- **PyPDFLoader:** PDF içerisindeki metni okumak.
- **RecursiveCharacterTextSplitter:** Uzun metinleri chunk'lara ayırmak.
- **OpenAI Embeddings:** Chunk'ları ve kullanıcı sorularını vektöre dönüştürmek.
- **Qdrant:** Vektörleri saklamak ve similarity search yapmak.
- **OpenAI LLM:** Bulunan bağlama göre cevap üretmek.
- **Gradio:** Kullanıcının soru sorabileceği web arayüzünü oluşturmak.

Docker Mimarisi

Projede iki ayrı Docker container kullandık.

rag-app Container

Bu container içerisinde;

- Python
- LangChain
- PDF Loader
- Text Splitter
- OpenAI Embedding
- Retriever
- OpenAI SDK
- Gradio

çalışmaktadır.

Qdrant Container

Bu container ise;

- Chunk vektörlerini
- Chunk metinlerini
- Sayfa bilgilerini
- Metadata bilgilerini

saklamaktadır.

Docker Compose bu iki container'ı aynı ağ üzerinde çalıştırdığı için uygulama Qdrant'a

```
http://qdrant:6333
```

adresi üzerinden erişebilmektedir. Container kullandığımız için burada **localhost** kullanılmamıştır.

Proje Dosya Yapısını Oluşturduk

İlk olarak proje klasörünü hazırladık.

```
pdf-rag-qdrant/

data/
    document.pdf

app/
    config.py
    ingest.py
    rag.py
    ui.py

Dockerfile
docker-compose.yml
requirements.txt
.env
```

Daha sonra her dosyanın görevini belirledik.

requirements.txt

Projede kullanacağımız bütün kütüphaneleri ekledik.

.env

API anahtarları, model isimleri ve Qdrant ayarlarını burada tuttuk.

config.py

.env içerisindeki ayarları merkezi olarak yönettik.

Dockerfile

Python uygulamasının Docker image'ını oluşturduk.

docker-compose.yml

rag-app ve qdrant servislerini aynı anda ayağa kaldıracak yapıyı hazırladık.

Docker Ortamını Kurduk

Ardından Docker ortamını hazırladık.

```
docker compose up -d
```

komutuyla;

- rag-app
- qdrant

container'larını çalıştırdık.

Qdrant Dashboard üzerinden collection oluşup oluşmadığını kontrol ettik. Ayrıca Docker kurulumu sırasında karşılaşılan image indirme ve container yönetimi gibi problemleri çözerek ortamı çalışır hâle getirdik.

Ingestion (Bilgi Tabanı Oluşturma)

Daha sonra sistemin en önemli kısmı olan **ingestion** aşamasını geliştirdik.

Bu aşamada;

```
PDF

↓

PyPDFLoader

↓

Document

↓
```

```
RecursiveCharacterTextSplitter

↓

Chunk

↓

OpenAI Embedding

↓

Qdrant
```

akışı çalıştı.

PDF içerisindeki metin okunarak küçük chunk'lara ayrıldı.

Her chunk embedding modeliyle vektöre dönüştürüldü.

Oluşturulan;

- chunk metni
- embedding vektörü
- sayfa numarası
- dosya adı
- chunk numarası

Qdrant veritabanına kaydedildi.

Bu işlem yalnızca PDF sisteme ilk kez eklenirken bir defa çalıştırıldı.

Retrieval (Soru-Cevap Akışı)

Bilgi tabanı hazırlandıktan sonra kullanıcı tarafındaki RAG akışını geliştirdik.

Kullanıcı soru sorduğunda sistem şu sırayla çalışmaktadır.

```
Kullanıcı Sorusu

↓

Embedding

↓
```

```
Qdrant Similarity Search

↓

En ilgili Chunk'lar

↓

Context Oluşturma

↓

Prompt Hazırlama

↓

OpenAI LLM

↓

Cevap

↓

Kaynak Sayfalar
```

Burada dikkat edilmesi gereken nokta, LLM'in PDF'yi doğrudan okumamasıdır. Önce Qdrant en alakalı chunk'ları bulur, ardından sadece bu bilgiler LLM'e gönderilir ve cevap oluşturulur. Böylece hem daha doğru hem de daha hızlı sonuç elde edilir.

Gradio Arayüzünü Oluşturduk

Daha sonra kullanıcı arayüzünü hazırladık.

Arayüz oldukça sade tutuldu.

Kullanıcı yalnızca;

- soru kutusu,
- gönder butonu,
- cevap alanı,
- kaynak sayfa bilgisi

görmektedir.

PDF yükleme alanı özellikle eklenmedi. Çünkü PDF'ler geliştirici tarafından daha önceden sisteme yüklenmişti.

Sistemi Test Ettik

Sistem tamamlandıktan sonra restoran menüsü PDF'si üzerinde kapsamlı testler yaptık.

Toplam **50 soru** hazırladık.

Sorular;

- Kolay
- Orta
- Zor
- Muhakeme
- Hallucination

olmak üzere beş kategoriye ayrıldı.

Test sonucunda;

- 43 soru doğru,
- 5 soru eksik,
- 2 soru yanlış

cevaplandı.

Genel başarı oranı **%86** olarak ölçüldü.

Hallucination testlerinde sistem, PDF'de bulunmayan bilgileri büyük ölçüde uydurmadan cevap verebildi. Ayrıca retrieval ayarları, chunk boyutu, overlap ve reranking gibi alanlarda yapılabilecek geliştirmeleri de değerlendirdik.

Çoklu PDF Desteğine Geçtik

Projenin son bölümünde sistemi yalnızca tek PDF ile sınırlı bırakmadık.

Artık `data` klasörüne 10–15 farklı PDF eklenebilecek.

Bu aşamada;

```
PDF1
PDF2
PDF3
...
PDF15
↓
DirectoryLoader
↓

Chunk
```

```
↓
Embedding

↓
Qdrant
```

şeklinde çalışacak.

Her chunk'ın metadata kısmına;

- kaynak PDF adı,
- sayfa numarası,
- chunk numarası

eklenecek.

Böylece kullanıcı farklı konularda soru sorduğunda sistem önce hangi PDF'nin ilgili olduğunu belirleyecek, ardından yalnızca o PDF'deki ilgili chunk'ları LLM'e göndererek doğru kaynaktan cevap üretebilecektir.

Günün Sonunda Ortaya Çıkan Sistem

Günün sonunda Docker üzerinde çalışan, LangChain ile PDF'leri işleyen, OpenAI Embeddings ile vektör oluşturan, Qdrant'ta semantic search yapan ve Gradio üzerinden kullanıcıların yalnızca soru sorarak yüklenen PDF'lerden kaynak gösteren cevaplar alabildiği uçtan uca çalışan bir RAG sistemi geliştirdik. Ayrıca sistemi tek PDF'den çoklu PDF desteğine genişletecek mimariyi de planlayarak bilgi tabanının farklı dokümanlarla ölçeklenebilir hâle gelmesini sağladık.

Proje İstatistikleri

İstatistik	Değer
Toplam PDF Sayısı	**7**
Toplam Test Sorusu	**350**
PDF Başına Soru	**50**
Kolay Soru	**70** (7×10)
Orta Seviye Soru	**70** (7×10)
Zor Soru	**75** (5 PDF'de 15, 2 PDF'de 10)
Reasoning / Muhakeme Sorusu	**70** (7×10)
Hallucination Sorusu	**65** (5 PDF'de 5, 2 PDF'de 10)

RAG PDF PROJE AKIŞI

1. GELİŞTİRİCİ TARAFI – INGESTION (BİLGİ TABANI OLUŞTURMA)

PDF Dosyaları
(1 veya Birden Fazla)
data/ klasörü

PDF Okuma
(PyPDFLoader)
Metin Çıkarılır

Chunk'lara Bölme
(Text Splitter)
Metin Parçalara Ayrılır

Metadata Oluşturma
- Sayfa Numarası
- Kaynak PDF Adı
- Chunk Numarası
- Diğer Bilgiler

Metadata Eklenir

OpenAI Embeddings
Her Chunk Vektöre Dönüştürülür

Vector + Chunk + Metadata
Örnek Kayıt:

```
Vector: [0.12, -0.45, 0.87, ...]
Payload:
{
  "text": "PDF içindeki metin...",
  "page": 8,
  "source": "document.pdf",
  "chunk_id": 21
}
```

Qdrant Vector Database
(Docker Container)
- Vector
- Chunk Metni
- Metadata
- Sayfa Bilgisi
- Kaynak PDF Adı

Vektörler Kaydedilir

SİSTEM KULLANIMA HAZIR

2. KULLANICI TARAFI – SORU CEVAP (RETRIEVAL)

Kullanıcı
(Gradio UI)
Soru Yazar

Soru Embedding'i Oluşturulur
Soru Vektöre Dönüşür

Qdrant'ta Similarity Search
En Benzer Chunk'lar Bulunur

İlgili Chunk'lar Getirilir
0.92
0.89
0.85
...
En Uygun Parçalar Seçilir

Context Oluşturma & Prompt Hazırlama
Soru + Context Birleştirilir

LLM (OpenAI GPT-4o-mini)
Cevap Üretir
Sadece Verilen Context'e Göre Cevap Üretir

Cevap Kullanıcıya Gönderilir
Cevap Ekranda Gösterilir

Kaynaklar Gösterilir
Sayfa Numarası ve Kaynak PDF Bilgisi Gösterilir

3. DOCKER MİMARİSİ

Docker Compose

rag-app Container
- Python Uygulaması
- LangChain
- OpenAI SDK
- ingest.py (Ingestion)
- rag.py (Retrieval)
- ui.py (Gradio Arayüzü)
- config.py (Ayarlar)

Aynı Docker Ağı Üzerinden Haberleşir
http://qdrant:6333

qdrant Container
- Vector Database
- Chunk Vektörleri
- Chunk Metinleri
- Metadata
- Port: 6333

Test edilen dokümanlar

- Atatürk
- İstanbul
- Osmanlı Devleti
- Klasik Türk Mutfağı
- Türk Devrimi ve Kurtuluş Savaşı
- Endemik Bitkiler
- Türk Halk Oyunları Makalesi

Projede kullanılan teknolojiler

- **Docker**
- **Qdrant Vector Database**
- **LangChain**
- **OpenAI Embeddings**
- **OpenAI GPT-4o-mini**
- **Gradio**
- **PyPDFLoader**
- **RecursiveCharacterTextSplitter**
- **QdrantVectorStore**
- **Similarity Search (Retrieval)**
- **Python**

Ölçülen performans kriterleri

- Basit Retrieval
- Detaylı Retrieval
- Uzun metinden bilgi çıkarma
- Reasoning (Muhakeme)
- Hallucination (Bilgi uydurma kontrolü)

Bu projenin kısa özeti ise:

- **7 farklı PDF** kullanıldı.
- **350 adet test sorusu** hazırlandı.
- Sorular **5 farklı zorluk seviyesinde** (Kolay, Orta, Zor, Reasoning, Hallucination) değerlendirildi.
- Amaç, çok dokümanlı bir RAG sisteminin **retrieval doğruluğunu**, **cevap kalitesini** ve **halüsinasyon üretme eğilimini** ölçmekti.

RAG PDF Test Sonuçları

Doküman	Doğru	Başarı	Hallucination
İstanbul PDF	**48/50**	**%96**	**1**
Atatürk PDF	**46/50**	**%92**	**1**
Halk Oyunları Makalesi	**46/50**	**%92**	**0**
Osmanlı Devleti PDF	**45/50**	**%90**	**0**
Türk Devrimi ve Kurtuluş Savaşı PDF	**44/50**	**%88**	**0**
Türk Mutfağı PDF	**43/50**	**%86**	**0**
Restoran Menü PDF	**43/50**	**%86**	**0**
Endemik Bitkiler PDF	**42/50**	**%84**	**0**
Çanakkale PDF	**37/50**	**%74**	**0**

Genel İstatistik

- **Toplam test edilen doküman:** 9
- **Toplam soru:** 450
- **Toplam doğru cevap:** 394
- **Genel başarı oranı: %87,56**
- **Toplam Hallucination: 2**